

ACCOUNT-HOLDER VERIFICATION

BioPrint

Recognize the behavior behind the password

An evidence-led design for adaptive typing–pointer verification

Abstract. BioPrint adds behavioral verification to password login: typing provides the first identity signal, device fingerprinting supplies context, and uncertain attempts receive a pointer or keypad challenge. We develop this design through public-dataset comparisons, paired-modality studies and controlled synthetic login experiments. A trained pointer encoder reduces synthetic false acceptance from **14/90 to 9/90**, with a corresponding increase in genuine rejection from **13/60 to 19/60**. Typing experiments show improved separation with richer enrollment and nonlinear models. The resulting system combines inexpensive initial checks with targeted additional evidence, while preserving explicit model compatibility and a reproducible demonstration of both successes and failures.

1 Introduction: verifying the operator, not only the secret

A correct password proves knowledge of a secret; it does not distinguish its owner from another person who has obtained it. Device recognition addresses a different question: whether the browser and computer are familiar. An impostor on the owner’s laptop may have a perfect device match, while the legitimate owner may arrive on a new phone. BioPrint treats these as complementary signals rather than allowing a familiar device to substitute for behavioral identity.

Behavioral biometrics analyzes how a person interacts: the durations and overlap of keystrokes, movement toward targets, and the timing of visual search and selection. Commercial systems such as BioCatch use physical and cognitive behavior in fraud detection [2]. The National Institute of Standards and Technology (NIST) distinguishes such fraud indicators from authentication factors [1]. BioPrint applies this principle to an inspectable login prototype; it is an additional account-risk layer, not a claimed replacement for cryptographic authentication.

1.1 The design argument

The goal is to reduce impostor access without challenging every genuine login. Typing is already present in password entry, so it is the natural first behavioral measurement. Pointer motion offers a different observation when typing is inconclusive. Device fingerprinting helps decide when stronger evidence is needed, while bot checks identify scripted or mechanically repeated input separately. This architecture is preferable to a single rigid threshold for the problem studied: it can retain an efficient familiar-user path, request more evidence near uncertainty, and expose which signal caused a decision.

The research asks three connected questions. Which representations improve each signal over a defensible baseline? Does combining signals actually improve verification when they are measured together? Which combination provides a useful security–usability tradeoff in the implemented login? Each experiment below answers one of these questions; the final design follows the evidence rather than assuming that more features or a more complex model must be better.

1.2 Terms and evaluation language

A *genuine* attempt comes from the enrolled owner; an *impostor* attempt comes from a different operator claiming that account. The following terms separate model development from evaluation.

Term	Definition used throughout this report
FAR / FRR	False acceptance rate / false rejection rate: the fraction of impostor / genuine attempts incorrectly admitted / rejected.
EER / pp	Equal error rate: approximate equality of FAR and FRR in a threshold sweep. Percentage points: an absolute difference between percentages.
TRAIN / DEV	Training-side fitting, selection and calibration data / development-evaluation data scored after freezing a candidate. Reused DEV is exploratory, not an untouched test.
Enrollment support	Known-owner recordings that construct a personal profile, separate from evaluated attempts.
Step-up verification	Additional interaction before an uncertain login is accepted. Requesting it is not itself a false rejection.

2 Final methodology: an adaptive typing–pointer login

2.1 Typing: a personal profile with a compatible learned scorer

For key position i , let $t_i^\downarrow$ and $t_i^\uparrow$ denote its press and release times, in milliseconds. BioPrint extracts hold duration $H_i = t_i^\uparrow - t_i^\downarrow$, down–down (press-to-press) interval $DD_i = t_{i+1}^\downarrow - t_i^\downarrow$, and up–down (release-to-next-press) interval $UD_i = t_{i+1}^\downarrow - t_i^\uparrow$. A negative UD_i records overlapping presses. One warmup and ten counted password repetitions establish a personal timing profile.

The initial scorer measures scaled distance from the owner’s median timings, using enrollment variability to avoid treating every deviation equally. The feature schema is the ordered timing-feature list. With other-user examples and separate calibration data in that schema, the code selects a **radial basis function support vector machine (RBF SVM)**. A support vector machine learns a separating boundary between owner and other-user timing vectors; its radial basis function compares nearby timing patterns nonlinearly, allowing interactions that a single average distance cannot capture [4]. The error penalty is 10; the kernel-width factor is 0.1 relative to feature count.

Let d_k be the typing anomaly score (larger means less owner-like), τ_k its acceptance threshold, and $r_k = d_k/\tau_k$ the normalized typing score; subscript k denotes keystrokes. The score is not a probability. On a familiar device, $r_k \leq 0.75$ permits direct login; available typing above $2\tau_k$ blocks. Otherwise, an enrolled additional check is requested. The supported learned-model password is the public Carnegie Mellon University (CMU) benchmark text `.tie5Roan1`. Other passwords retain their personal distance profile until compatible labeled background and calibration data exist. More repetitions alone do not create that bank or automatically establish a validated new model.

2.2 Device fingerprinting and bot checks

The device profile compares hardware, browser and network attributes individually: screen characteristics, fonts, operating-system family, processor/memory characteristics, graphics family, language, browser version, rendering signatures and network context. Stable majority values across enrollment form the reference. Unstable attributes are omitted; changes are weighted rather than requiring an exact fingerprint hash match.

$$D_{\text{dev}} = G_h + 0.2G_b + 0.5G_n.$$

Here D_{dev} is the device-disagreement score, and G_h , G_b , G_n are sums of weighted attribute differences in the hardware, browser and network groups. The weights are heuristic information-based weights, not calibrated identity probabilities. A score above 6 indicates a changed device/context. Lower browser and network weights tolerate ordinary updates or movement between networks. A changed device requests stronger behavioral evidence; it is not itself proof of an impostor. Automation indicators, event trust and replay/timing checks remain a separate bot signal.

2.3 Trained pointer verification

Each account records three one-minute mouse/trackpad sessions. The encoder receives standardized absolute coordinate displacements in blocks of 128. A frozen **fully convolutional network (FCN)** converts each block into a vector of 128 learned features. The owner’s template averages normalized enrollment vectors; the challenge is matched by mean cosine similarity:

$$\hat{z}_j = \frac{z_j}{\|z_j\|_2}, \quad c_u = \frac{\sum_{j \in E_u} \hat{z}_j}{\|\sum_{j \in E_u} \hat{z}_j\|_2}, \quad s_p = \frac{1}{5} \sum_{j=1}^5 \hat{z}_j^\top c_u.$$

Here u identifies the account; E_u is its set of enrollment blocks; j indexes blocks; z_j is an encoder vector; $\hat{z}_j$ is its unit-length form; $\|\cdot\|_2$ is Euclidean length; c_u is the unit-length personal template; and $\top$ forms the dot product. The last sum uses the first five *challenge* blocks, and s_p is pointer similarity. Acceptance requires $s_p \geq 0.948082149$. Five blocks require 641 coordinates, with no padding or cross-session concatenation. The three-second minimum is only a lower bound; source-data median capture time was 16.89 seconds. The encoder uses the same normalization and threshold as the source experiment.

2.4 Challenge routing and application behavior

Uncertain desktop attempts use the trained pointer check after pointer enrollment; the existing motor profile remains a fallback. On a phone, two scrambled-keypad runs compare search/cadence against a profile built from six enrollment runs. Current mobile acceptance uses the full personal threshold. Nonces, expiry, account-bound captures and session-cookie checks connect scoring to real access decisions. The application programming interface (API), implemented with FastAPI and SQLite, preserves raw recordings and separate profiles. An invitation-protected encrypted local-network gateway supports cross-device testing. Accepted attempts do not silently update the owner’s template.

3 Experimental setup: data, baselines and improvements

3.1 What was measured and what changed

The comparison unit is an account claim: a recorded interaction scored against a claimed user’s support profile. Table 1 describes the data and the actual methodological change behind each main result. *ExtraTrees* means extremely randomized trees, an ensemble with randomized split choices; a random forest combines decision trees fitted to sampled data. Both can model nonlinear interactions. Logistic regression is the linear classification comparator.

Dataset / task	Data and evaluation support	Baseline → candidate: why test it?
CMU password timing	Fixed password press/release timing; 51 accounts; ten enrollments, with a separate 100-enrollment budget	Scaled Manhattan distance → per-account RBF SVM. Learn nonlinear owner/background boundaries; select settings with separate sessions.
KeyRecs fixed text	Repeated password timing; 79 accounts and 7,896 development observations	Logistic regression on 47 positional features → ExtraTrees on the same features, isolating classifier improvement.
KeyRecs free transcription	Variable prompted text; 78 probe accounts, 4,525 windows of 50 adjacent key pairs	Logistic regression on 35 timing summaries → ExtraTrees. Model nonlinear relationships without a fixed key sequence.
SapiMouse controlled pointer	Browser mouse trajectories; 24 development users; 76 genuine/1,748 impostor claims	Handcrafted scaled distance → FCN embeddings with personal cosine matching. Learn movement structure at the same five-block budget.
Balabit pointer	Remote-desktop movement logs; nonoverlapping 128-movement windows; session-separated evaluation	Handcrafted scaled distance → ExtraTrees over 29 movement descriptors. Test whether nonlinear geometry generalizes across sessions.
FPStalker browser linkage	Repeated browser attributes; 154 browser identities; 1,176 genuine/23,520 impostor pairs	Equal attribute agreement → histogram gradient boosting on attribute similarity. Learn which changes preserve browser identity.
DELBOT-Mouse bot detection	Human and automated mouse paths; held-out bot family; 459 human/55 bot traces	Pointer-only rule reconstruction → random forest on geometry. Test learned shape discrimination beyond fixed rules.
Stroop/Flanker cognition	Response times and accuracy under different task conditions; nine development people	A condition-index timing slope → a full reaction-time/accuracy profile. Retain absolute speed, condition detail and errors.
Touch Sensing Images (TSI)	Touchscreen target/landing measurements; 12 development people and 356 trials	Landing baseline → random forest using target-normalized landing, movement and timing. Account for target geometry.

Table 1: Matched feature budgets or explicit adaptations make the comparison interpretable. Browser linkage and bot detection are different tasks from person verification.

3.2 Fitting and threshold control

The ten-enrollment CMU study uses session 1 for enrollment, sessions 2–3 for parameter selection, session 4 for threshold calibration and sessions 5–6 for evaluation. Nine combinations of support-vector penalty and kernel width are compared. KeyRecs splits session 1 chronologically into 50% fitting, 25% selection and 25% calibration, then evaluates session 2. Pointer and other datasets use their recorded participant/session partitions, preserving support/probe separation and source hashes. Thresholds target 1% empirical calibration FAR unless specified otherwise; the achieved development FAR may differ.

For threshold τ , $\text{FAR}(\tau) = N_{FA}/N_I$ and $\text{FRR}(\tau) = N_{FR}/N_G$, where N_{FA} and N_{FR} count false accepts and false rejects, while N_I and N_G count impostor and genuine claims. CMU and the original KeyRecs studies average per-account EER; other main comparisons pool claims. An EER improvement is therefore interpreted within its protocol, not as a universal ranking across datasets. Repeated claims share participants and are not independent people.

3.3 Combining evidence without inventing real identities

Four isolated branches replay the same 39 artificial profiles and 390 scenarios each: 12 TRAIN, 12 calibration and 15 DEV accounts, totaling 1,560 API scenarios. Development contains 60 genuine/90 impostor attempts. CMU provides typing; the initial cohort simulates targets, context and mobile input. The trained-pointer follow-up preserves these cases but maps accounts to 15 distinct SapiMouse users with seed 20260920, using their separate support and native probe recordings. These are controlled artificial pairings, not linked real people. Paired gameplay data provide a separate check on genuine cross-modality transfer (Section 4.3). Source hashes, invalid-window accounting, API parity and saved early failures keep each result reproducible. Follow-ups on inspected development data are identified as exploratory.

4 Results: what improves, why it improves, and what it enables

4.1 Matched dataset comparisons

Every entry in the next table is **FAR / FRR / EER (%)**. The final column is baseline EER minus candidate EER, in percentage points; positive values indicate improvement. Table 1 explains the representation, baseline and candidate behind each row.

Dataset / task	Baseline	Candidate	EER gain (pp)
CMU, 10 enrollments	1.10 / 74.20 / 22.04	0.95 / 67.43 / 14.32	+7.72
CMU, 100 enrollments	1.01 / 49.78 / 13.99	1.03 / 44.43 / 9.14	+4.85
KeyRecs fixed text	1.26 / 76.39 / 17.08	1.29 / 45.91 / 11.62	+5.46
KeyRecs free text	0.70 / 85.73 / 16.66	1.14 / 45.64 / 9.72	+6.94
SapiMouse, five blocks	8.30 / 60.53 / 26.17	1.66 / 43.42 / 13.87	+12.30
FPStalker browser linkage	0.69 / 8.33 / 3.79	0.84 / 4.85 / 2.30	+1.49
DELBOT bot geometry	0.00 / 100.00 / 50.50	3.64 / 31.15 / 14.55	+35.95
Stroop/Flanker cognition	1.39 / 88.89 / 48.61	2.78 / 77.78 / 27.78	+20.83
TSI touch landing	1.00 / 95.51 / 36.75	1.00 / 91.85 / 34.45	+2.30
Balabit pointer	1.89 / 98.20 / 48.61	1.93 / 92.45 / 56.18	-7.58

Table 2: Measured improvements and tradeoffs under each dataset’s protocol. Lower is better, but an EER gain need not lower both operating errors.

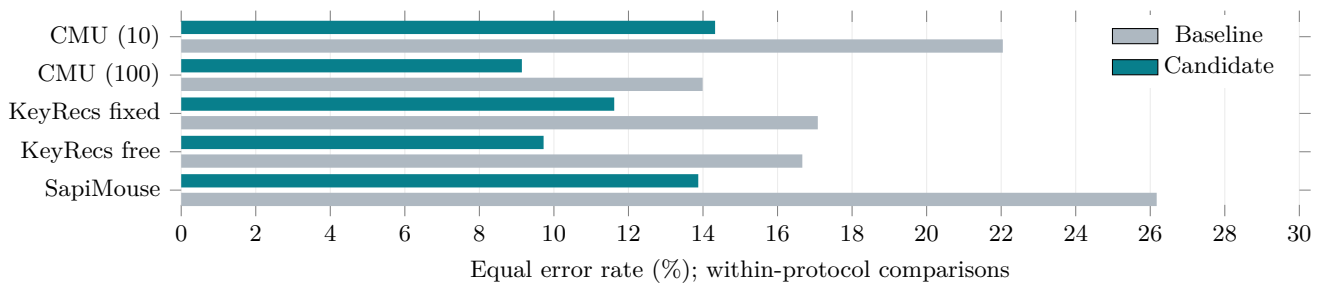


Figure 1: Nonlinear typing models and learned pointer representations improve separation. The 100-enrollment CMU result explicitly uses more owner data.

Typing improves through representation, model choice and support. With ten CMU enrollments, account-specific SVM selection reduces EER from 22.04% to 14.32% (35.02% relative reduction). The 95% bootstrap interval, obtained by resampling accounts, for new minus old EER is $[-11.11, -4.55]$ pp. KeyRecs keeps its feature sets fixed and changes the classifier: fixed/free transcription FRR falls by 30.47/40.08 pp. This supports a learned nonlinear scorer when representative same-schema data are available, rather than treating all timing deviations as equally informative.

More enrollment helps in the measured CMU comparisons. At 100 repetitions, SVM EER is 9.14%, compared with 14.32% for the ten-repetition account-selected model; FRR falls from 67.43% to 44.43% at approximately 1% FAR. The baseline also improves with more support (22.04% to 13.99% EER). These are separately selected support budgets, not a fixed-model learning curve; they support richer personal reference data without guaranteeing improvement for every password. The deployment strategy is consequently progressive: start with the personal distance profile, use the compatible learned path when its labeled reference and calibration bank exist, and retain step-up verification for uncertainty.

Pointer learning adds useful structure. At the same 641-coordinate budget, FCN-cosine reduces false accepts from 145/1,748 to 29/1,748 and false rejects from 46/76 to 33/76. The absolute EER gain is 12.30 pp. Alternative one-class support vector machines give 2.23/80.26/21.28 (raw scores) and 2.40/61.84/18.36 (enrollment-normalized scores). Background Z-normalization, which standardizes against background score statistics, gives 0.74/60.53/11.73; it reduces false acceptance but rejects more owners and was not selected by the training objective. Its EER-change interval includes zero.

Context and cognition support selective use. Learned FPStalker linkage improves EER to 2.30%, while exact fingerprint equality rejects 91.33% of genuine pairs. This supports tolerant attribute comparison, although the live app uses its own weighted rule rather than that learned classifier. Full cognitive profiles outperform a single slope because they retain information; their 77.78% FRR and long tasks make them weaker login candidates. Touch gains are modest, and Balabit’s pooled EER worsens. The evidence favors integrating the transferable typing and controlled-pointer components while using other signals for their demonstrated roles.

4.2 Typing-model diagnostics and a reproducible demonstration

The compatible SVM was used in the synthetic benchmarks. A read-only diagnostic isolates its initial model score from bot, device and step-up decisions. Across 45 genuine desktop and 30 ordinary impostor desktop scenarios, it accepts 1 impostor and rejects 24 owners: **FAR 3.33%, FRR 53.33%**. Including all 60 impostor scenarios with typing, including owner-like and scripted inputs, gives $8/60 = 13.33\%$ FAR; FRR remains 24/45. This post-hoc diagnostic of the existing cohort explains why uncertain typing needs another modality; it is not an independent new benchmark.

One script fits the compatible model from temporary enrollments and calls the real API, using public text `.tie5Roan1`. A true accept/reject is a correct owner/impostor decision; a false reject/accept is the corresponding error. These freshly reproduced examples are deliberately selected, not an accuracy sample. Model acceptance uses 1.0; direct admission uses 0.75.

Illustrated typing outcome	Score	Model limit	Actual login action
True accept: owner matches	0.0000	1.0000	Admit
True reject: impostor mismatches	3.5616	1.0000	Block
False reject: owner mismatches	4.1786	1.0000	Block
False accept: impostor matches	0.7656	1.0000	Request keypad; no session
Owner-like typing attack	0.0000	1.0000	Admit (deliberate attack case)

Table 3: The stricter policy can intercept a typing-model false accept. An owner-like replay assumption still exposes a direct-admission weakness. Neither score is fabricated.

Run `bash .worktrees/typing-pointer/experiments/typing_demo/run.sh` from the repository root. The script checks model use, saved-score agreement and session issuance, then opens a local browser report. It requires no live accounts or network listener.

4.3 Why evaluate general models and genuinely paired fusion?

General typing tests whether the fixed-password restriction can be removed. Shared logistic and ExtraTrees models use 42 distribution-residual features, followed by 31 features aligned to each account’s own password. Fifty-four evaluations cover unseen users and cross-dataset transfer. Aligned logistic, trained on both sources, yields 0.42/95.30/25.41 on CMU and 3.83/88.98/29.42 on KeyRecs fixed. Trees also retain high rejection (90.90% and 82.61%). Matched shipped-scorer EERs are 26.31% and 31.64%. These rejection rates favor retaining compatible-password models. Distribution-only models perform worse overall; free-text TypeNet transfer gives 1.17/84.08/20.23.

BEACON tests whether promising encoders combine on the same people. The Behavioral Engine for Authentication and Continuous Monitoring (BEACON) records gameplay keyboard, mouse and hardware signals [5]. Our subset has three development participants, 81 genuine and 162 impostor claims. TypeNet and Type2Branch are keystroke-sequence encoders; SapiMouse supplies pointer embeddings. Their scores and handcrafted features feed logistic fusion: 24 candidates combine eight feature sets with three regularization strengths. Separate training identities select and calibrate the model. All 81 short windows remain; no cross-dataset identity links are invented.

Paired BEACON candidate	FAR (%)	FRR (%)	EER (%)
Handcrafted features only	2.47	97.53	42.90
TypeNet + SapiMouse	54.94	59.26	59.26
TypeNet + SapiMouse + handcrafted	45.68	30.86	38.27
Type2Branch + SapiMouse + handcrafted	24.69	67.90	40.74

Table 4: Fusion has no automatic advantage under task transfer. The last change prevents 34 false accepts but creates 30 more false rejects.

Earlier handcrafted fusion gives 2.47/98.77/44.14; adding context gives 8.02/86.42/43.21. Nonlinear search retains the linear fusion model; alternative neural selection gives 4.32/92.59/43.21. These studies test an alternative architecture rather than validate our login. Their weak cross-task transfer supports an adaptive policy with task-compatible components over deploying an unvalidated universal fusion model.

Other experiments test feasibility. A short-capture Type2Branch adaptation covers 79 accounts but gives 9.14/30.38/13.93, missing the calibration target. Hand Movement, Orientation and Grasp (HMOG) combines typing with an inertial measurement unit (IMU); joint-model rates are 0/100/45.16 with only 20.13% probe coverage. Three-level arithmetic has 11/12 calibration genuine rejections and an invalid development block; the HMOG touch extension and original long-capture Type2Branch protocol lack required support. BrainRun establishes a schema audit only (101 games, 982 gestures). These results defer brief cognition and sensor fusion; all variants and infeasible statuses remain in the ledger.

4.4 Choosing the login combination

All policies face the same 60 genuine and 90 impostor development scenarios. Candidates share compatible learned typing but request different checks when evidence is uncertain. The pointer follow-up replaces statistical movement scoring with the actual trained encoder on mapped source recordings.

Policy / verification method	False accepts	FAR (%)	False rejects	FRR (%)
Control	32/90	35.56	5/60	8.33
Typing repetition	10/90	11.11	25/60	41.67
Typing + statistical pointer	14/90	15.56	13/60	21.67
Typing + keypad	13/90	14.44	16/60	26.67
Typing + trained pointer	9/90	10.00	19/60	31.67

Table 5: Final synthetic access outcomes after prescribed verification. The original candidates all missed the 1% calibration gate; the prototype choice is a security-prioritized tradeoff, not a certified low-FAR operating point.

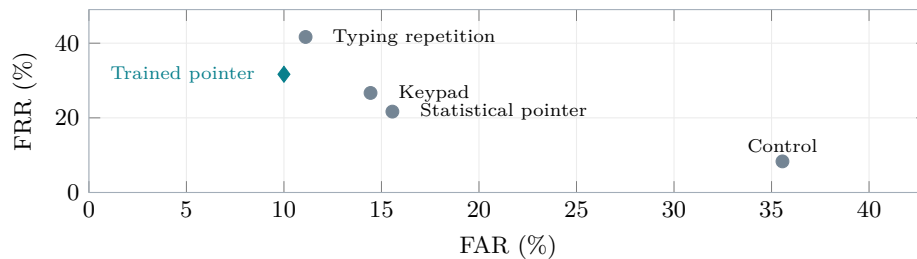


Figure 2: The measured operating tradeoffs. Lower left is preferable. These are conditional policies, not points on a common single-score threshold curve; no app-wide EER is inferred.

Why choose trained pointer? Its synthetic FAR is lowest (10.00%); FRR is below typing repetition (31.67% versus 41.67%). Against statistical pointer, it prevents five false accepts and adds six net genuine failures. Both challenge 63/150 cases; the encoder decides 33 desktop cases. All 33 scores agree with frozen predictions within 0.000002, and all 300 paired session outcomes were checked. This is a concrete gain for prioritizing unauthorized-access reduction.

Is either pointer method statistically superior? No overall winner is established. An exploratory paired sign-flip test swaps method labels for whole accounts over all 2^{15} assignments. Its two-sided FAR/FRR probability values, 0.125/0.156, exceed the conventional 0.05 significance level. They measure extremeness under an exchangeable-method null assumption, not the probability a model is correct. Statistical pointer makes 27 errors versus 28 for trained pointer. On these fixed counts, trained pointer has lower cost when a false accept costs more than 1.2 times a false reject. *Security priorities*, not statistical dominance, motivate the choice.

The nine residual false accepts bypass the encoder: seven owner-like typing attacks and two mobile cases. Across 100 mappings, FAR spans 10.00–13.33% and FRR 25.00–41.67%. Fixing an adapter that rejected valid pauses reduces FRR from 50.00% to 31.67%. Source-calibration targets 0.1/1/5% give app FRR 33.33/31.67/28.33%, with FAR 10%. The latest mobile timing-rule and threshold changes are functionally tested but postdate these aggregate results.

5 Conclusion

BioPrint turns behavioral recognition into an executable strategy: **recognize familiar behavior efficiently, use device context intelligently, and request a different signal when evidence is insufficient**. Dataset studies establish how nonlinear typing and learned pointer representations improve their baselines; paired studies establish why gains must be transferred selectively. The prototype connects that reasoning to enrollment, model inference and actual access decisions, with a reproducible demonstration of successes and failure modes.

The central contribution is targeted additional evidence, not indiscriminate feature accumulation. Trained pointer prevents five additional impostor admissions in the controlled comparison while preserving direct login for convincing typing. Residual attacks identify the next precise target: direct-admission verification. This combination of measured improvement, explicit tradeoffs and working integration provides a strong foundation for broader paired-user validation and progressively richer personal models.

References and reproducibility. [1] NIST, [Digital Identity Guidelines: Authentication and Authenticator Management](#). [2] BioCatch, [Behavioral biometrics and fraud detection](#). [3] Killourhy and Maxion, [CMU keystroke benchmark](#). [4] scikit-learn, [Support Vector Machines](#). [5] Singh et al., [BEACON dataset](#). Local evidence: `complete-results.csv`, `evidence-manifest.json`, `paired-comparison-analysis.json`, and the typing demo's `results.json`. Dataset-rate tables are generated from saved evidence; rates retain their stated protocols.